

CHAINMAILLE CALCULATION APPLICATION

By Kaitlyn Vinson

Senior Project

Kaitlyn Vinson
kmvinson@csustudent.net

Introduction/abstract

This application is to be used by chainmaille artists to assist them in calculating the exact quantity of rings to use for a project. The calculation portion of the application shows the user the open and closed rings that they will need so they can prepare the rings, making the weave more efficient. The application also allows users to keep an inventory of their rings, granting the user the ease of knowing they have enough rings for projects, or showing them when they need to order more. The users can update their ring counts to match their inventory or when they purchase more rings. The final page of the application contains instructions on how to do some common weaves to remind the user. Each instruction set has detailed images as well as written out steps on how to create the weave.

Background/why you did it

I created this application because I create chainmaille jewelry and needed a way to keep track of my inventory and be able to calculate the rings that I need. The calculations page allows me to prepare the number of rings that I will need for any given project and then subtract that from the inventory. I use the instruction page to remind myself how to create a weave that I have not done in a while. This application allows me to have everything that I or any chainmaille user will need in one application.

This application is needed because there are no other applications like it. There are other applications that contain instructions on weaves, and some tell the user what aspect ratio the user needs for a weave. The aspect ratio is used to determine what ring sizes can be used for weave so that it is not too tight or too loose. There are no other applications that calculate the actual ring count needed and no inventory applications for the amount of rings the user owns.

Goals and Accomplishments

Calculations Tab:

Goals: Have Dropdown menus to select the weave, ring gauge, ring size, and length and calculate the open and closed rings. Have a button that subtracts the rings of the project from the inventory. Be able to select a custom length that the user enters to calculate the rings.

Accomplishments: I was able to accomplish everything except for the custom length selection and an alert for when the user presses the subtract button.

Inventory Tab:

Goals: Create a database that stores the inventory of rings and can be updated and added to.

Accomplishments: I was able to accomplish everything except for the addition button. The user is still able to update the inventory manually, but I would like to add the addition button in the future.

Instructions Tab:

Goals: Create buttons that brings up pages that contain the instructions for different weaves.

Accomplishments: All of this goal was accomplished.

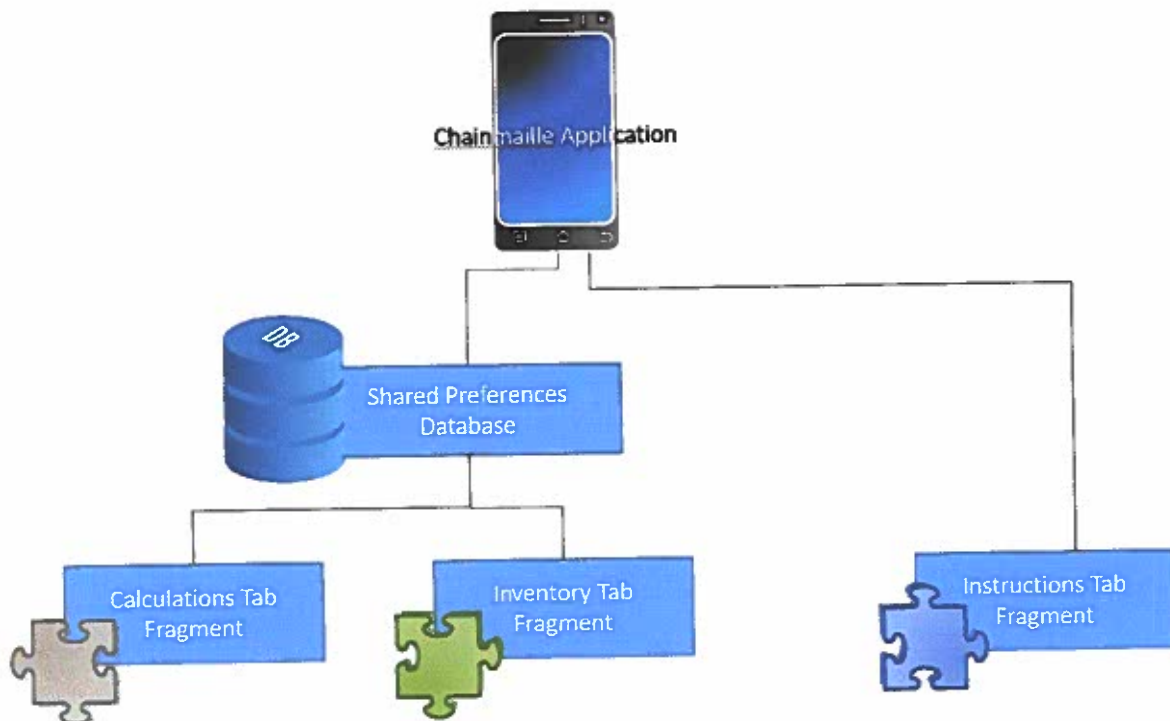
Tools Used

Android Studio: Used Java and HTML in Android studio to code the application. Also used the Junit library in Android to test the application and its functions. Used Fragments to create the tabs in the application and Shared Preferences as the database storage.

Samsung Galaxy S8: Used the phone to test the application on as well as the in application virtual phone that Android Studio Contains.

Overall Design

The application is designed using Java and HTML to create Fragments and a Shared Preferences Database. The Fragments are the tabs of the application and include the Calculations tab, the Inventory tab, and the Instructions tab. The Shared Preferences Database is connected to the Calculations tab and the Inventory tab. Shared Preferences is used to store the inventory of the rings so that the user can update the amount or subtract from it using the subtract button of the Calculations tab.



Testing

Junit testing is what was used in the application to test the accuracy of the Calculations tab.

Android studio contains its own way to test using Junit.

```
import static org.hamcrest.CoreMatchers.is;
import static org.junit.Assert.*;

public class calc_tabTest {
    calc_tab calc;
    @Before
    public void setUp(){
        calc = new calc_tab();
    }
    @Test
    public void calcTest1(){
        final int total = calc.calculateTotals("Spiral","16","1/4", 5);
        assertThat(total, is(35));
    }
    @Test
    public void calcTest2(){
        final int total = calc.calculateTotals("Spiral","18","1/8", 7);
        assertThat(total, is(0));
    }
    @Test
    public void calcTest3(){
        final int total = calc.calculateTotals("Box","16","3/8", 23);
        assertThat(total, is(322));
    }
    @Test
    public void calcTest4(){
        final int total = calc.calculateTotals("Box","20","1/8", 5);
        assertThat(total, is(250));
    }
    @Test
    public void calcTest5(){
        final int total = calc.calculateTotals("Shaggy Loops","16","7/32", 12);
        assertThat(total, is(144));
    }
    @Test
    public void calcTest6(){
        final int total = calc.calculateTotals("Shaggy Loops","18","1/8", 24);
        assertThat(total, is(0));
    }
    @Test
```

```
public void calcTest7(){  
    final int total = calc.calculateTotals("Shaggy Loops","19","3/16", 10);  
    assertThat(total, is(150));  
}}
```

Challenges Faced

Database Errors: I first tried to use a Database that was not Shared Preferences but was instead a normal database. This did not work, however, because the fragment code would interfere with the database. Instead I chose Shared Preferences which was better for the application because it does not take a lot of memory and is easier for the application to access between the fragments.

Code Placement: I ran into a few bugs that were caused by the incorrect order of code statements. I believed that there were other issues and spent time trying to discover those but stumbled upon the issue of the code not being in the correct order.

Accessing Data: In the beginning I ran into difficulties accessing the data in between the fragments. I eventually discovered a way to remedy the situation and was able to create a link between the fragments using Shared Preferences.

Setting up Testing: It took some trial and error to get the testing phase working. I ended up discovering that I had some Null Pointer Exceptions that I needed to fix, and once that was fixed I was able to proceed with my testing.

What I Learned

I learned that creating a functioning application takes a lot of work. If I could change anything, I would have started off with something other than Fragments as my tab system. There is not much support and information on how to do things with fragments and I may have been able to

implement a regular database and other features with more ease. In the future I would love to learn more on implementing more features.

Future Features

In the future I wish to add more weaves to the Calculation and the Instruction pages. I will also do more testing to ensure that all of the calculations are the best they can be. I will add an addition button to the inventory page so the user can add any rings they purchase.

I will also add another inventory page that keeps track of completed projects that the user can manually enter to fit their needs. Along with a new inventory page, I want to add calculations that give the cost to create the project so the user can decide how much to charge for their piece.

Chainmaille Calculation App

BY KATLYN VINSON

1

Description of Project

- ▶ Application on Android Phone
- ▶ Calculation Tab
- ▶ Inventory Tab
- ▶ Instruction Tab

2

Why I chose this project

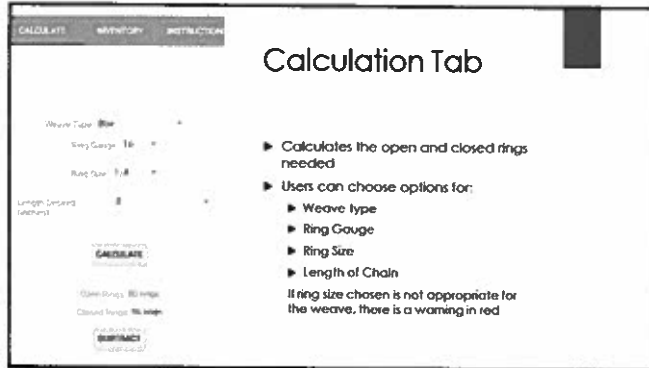
- ▶ Chainmaille Business
- ▶ Extensive Inventory
- ▶ No way to calculate rings for each project
- ▶ Many weave types to remember

3

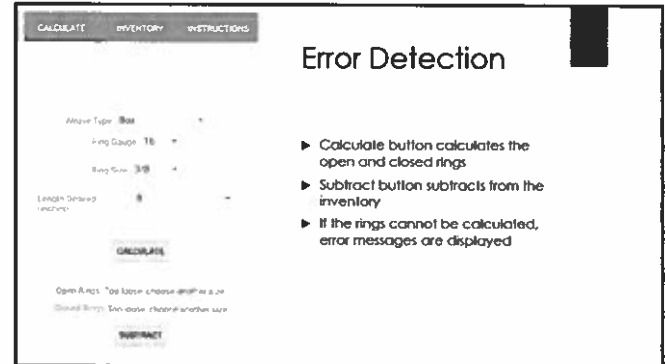
Testing

- ▶ Testing individual modules
 - ▶ Calculations – going through each ring size, ensuring the calculations are correct
 - ▶ Adding and subtracting from inventory – ensuring that the correct total is being subtracted from the inventory
 - ▶ Ensure instruction pages are correct – each link goes to the correct instruction page

4



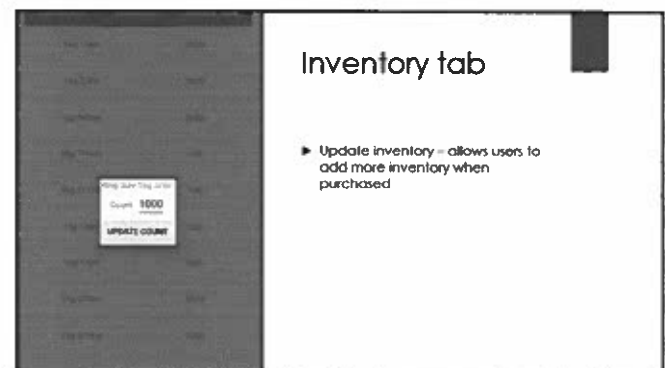
5



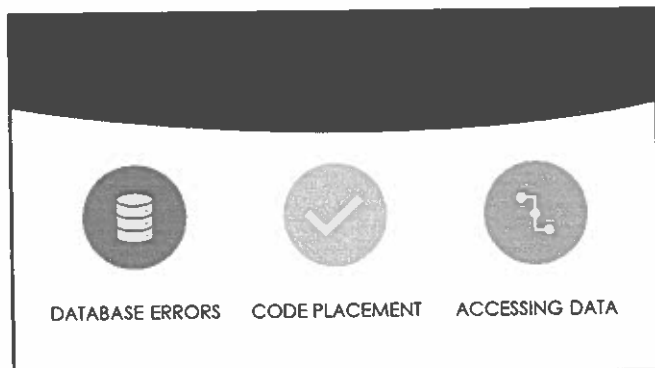
6



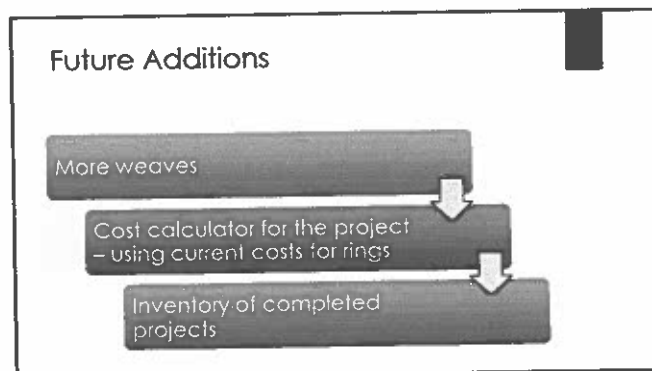
7



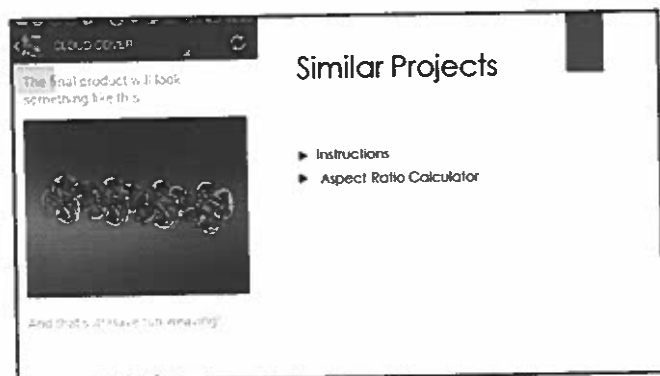
8



13



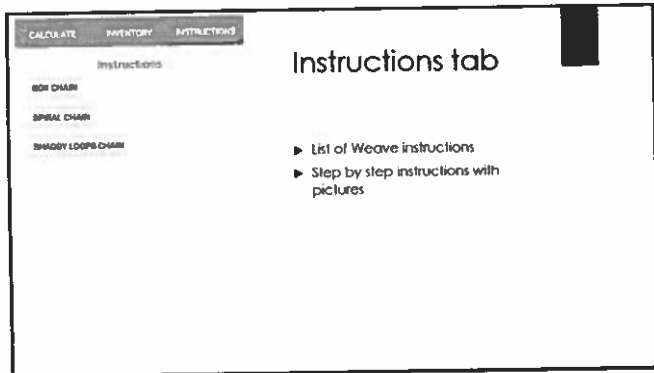
14



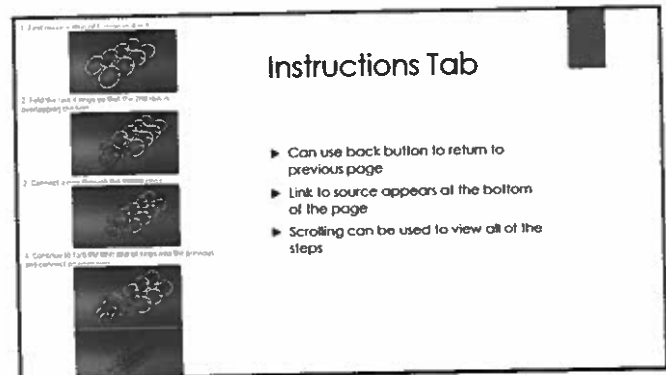
15



16



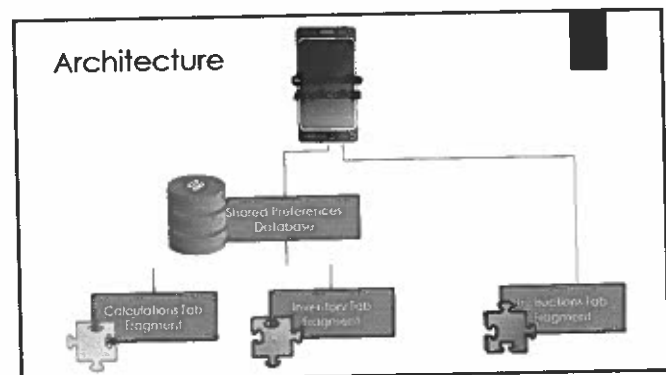
9



10



11



12

```
1  /* MainActivity.java
2  * Controls all of the application and sets up the
3  * fragment pages.
4  */
5  package c.kaitlyn.chainmaillecalculations;
6
7  import android.content.Context;
8  import android.content.DialogInterface;
9  import android.content.Intent;
10 import android.content.SharedPreferences;
11 import android.os.Bundle;
12 import android.support.annotation.NonNull;
13 import android.support.design.widget.BottomNavigationView;
14 import android.support.design.widget.FloatingActionButton;
15 import android.support.design.widget.TabLayout;
16 import android.support.v4.app.FragmentActivity;
17 import android.support.v4.content.res.TypedArrayUtils;
18 import android.support.v4.view.ViewPager;
19 import android.support.v7.app.AlertDialog;
20 import android.support.v7.app.AppCompatActivity;
21 import android.view.Menu;
22 import android.view.MenuItem;
23 import android.view.View;
24 import android.widget.AdapterView;
25 import android.widget.ArrayAdapter;
26 import android.widget.Button;
27 import android.widget.EditText;
28 import android.widget.LinearLayout;
29 import android.widget.Spinner;
30 import android.widget.TextView;
31 import android.graphics.Color;
32 import android.widget.Toolbar;
33
34 import com.google.gson.Gson;
35 import com.google.gson.reflect.TypeToken;
36
37 import java.lang.reflect.Type;
38 import java.util.ArrayList;
39
40 import c.kaitlyn.chainmaillecalculations.fragments.
    calc_tab;
41 import c.kaitlyn.chainmaillecalculations.fragments.
    instruction_tab;
42 import c.kaitlyn.chainmaillecalculations.fragments.
    inventory_tab;
```

```
43
44 public class MainActivity extends FragmentActivity {
45
46     //Code for tabs
47     private TabLayout tabLayout;
48     private ViewPager viewPager;
49     //Arraylist for inventory page
50     ArrayList<ringInventory> ringInvList = new ArrayList<>
    ();
51
52     @Override
53     protected void onCreate(final Bundle
savedInstanceState) {
54
55         super.onCreate(savedInstanceState);
56         setContentView(R.layout.activity_main);
57
58         loadData();
59
60         Bundle bundle = new Bundle();
61
62         //Code for tab
63         tabLayout = (TabLayout) findViewById(R.id.
tablayout_id);
64         viewPager = (ViewPager) findViewById(R.id.
viewpager_id);
65         ViewPagerAdapter adapter = new ViewPagerAdapter(
getSupportFragmentManager());
66         //Adding Fragments
67         adapter.AddFragment(new calc_tab(), "Calculate");
68         adapter.AddFragment(new inventory_tab(), "
Inventory");
69         adapter.AddFragment(new instruction_tab(), "
Instructions");
70
71         //adapter setup
72         viewPager.setAdapter(adapter);
73         tabLayout.setupWithViewPager(viewPager);
74         saveData();
75
76 }
77     private void saveData(){
78         SharedPreferences sharedPreferences = this.
getSharedPreferences("shared preferences", Context.
MODE_PRIVATE);
```

```
79         SharedPreferences.Editor editor =
sharedPreferences.edit();
80         Gson gson = new Gson();
81         String json = gson.toJson(ringInvList);
82         editor.putString("inventory", json);
83         editor.apply();
84     }
85
86     private void loadData(){
87         SharedPreferences sharedPreferences = this.
getSharedPreferences("shared preferences", Context.
MODE_PRIVATE);
88         Gson gson = new Gson();
89         String json = sharedPreferences.getString("
inventory", null);
90
91         Type type = new TypeToken<ArrayList<ringInventory
>>() {}.getType();
92         ringInvList = gson.fromJson(json, type);
93
94         if (ringInvList == null){
95             ringInvList = new ArrayList<>();
96             populatelist();
97         }
98     }
99 private void populatelist(){
100     ringInventory gauge1614 = new ringInventory("16g 1/
4in", "2400");
101     ringInventory gauge1638 = new ringInventory("16g 3/
8in", "1000");
102     ringInventory gauge16516 = new ringInventory("16g 5/
16in", "2000");
103     ringInventory gauge16716 = new ringInventory("16g 7/
16in", "100");
104     ringInventory gauge16732 = new ringInventory("16g 7/
32in", "300");
105     ringInventory gauge1814 = new ringInventory("18g 1/
4in", "200");
106     ringInventory gauge1818 = new ringInventory("18g 1/
8in", "600");
107     ringInventory gauge18316 = new ringInventory("18g 3/
16in", "5000");
108     ringInventory gauge18516 = new ringInventory("18g 5/
16in", "1000");
109     ringInventory gauge18532 = new ringInventory("18g 5/
```

```
109 32in", "500");
110     ringInventory gauge18732 = new ringInventory("18g 7/
    32in", "700");
111     ringInventory gauge1914 = new ringInventory("19g 1/
    4in", "200");
112     ringInventory gauge19316 = new ringInventory("19g 3/
    16in", "100");
113     ringInventory gauge19532 = new ringInventory("19g 5/
    32in", "300");
114     ringInventory gauge19964 = new ringInventory("19g 9/
    64in", "100");
115     ringInventory gauge2018 = new ringInventory("20g 1/
    8in", "1600");
116     ringInventory gauge20316 = new ringInventory("20g 3/
    16in", "1200");
117     ringInventory gauge20332 = new ringInventory("20g 3/
    32in", "1000");
118     ringInventory gauge20532 = new ringInventory("20g 5/
    32in", "750");
119     ringInventory gauge20564 = new ringInventory("20g 5/
    64in", "300");
120     ringInventory gauge20764 = new ringInventory("20g 7/
    64in", "400");
121     ringInventory gauge20964 = new ringInventory("20g 9/
    64in", "900");
122     ringInventory gauge201164 = new ringInventory("20g 11
    /64in", "300");
123
124     ringInvList.add(gauge1614);
125     ringInvList.add(gauge1638);
126     ringInvList.add(gauge16516);
127     ringInvList.add(gauge16716);
128     ringInvList.add(gauge16732);
129     ringInvList.add(gauge1814);
130     ringInvList.add(gauge1818);
131     ringInvList.add(gauge18316);
132     ringInvList.add(gauge18516);
133     ringInvList.add(gauge18532);
134     ringInvList.add(gauge18732);
135     ringInvList.add(gauge1914);
136     ringInvList.add(gauge19316);
137     ringInvList.add(gauge19532);
138     ringInvList.add(gauge19964);
139     ringInvList.add(gauge2018);
140     ringInvList.add(gauge20316);
```

```
141     ringInvList.add(gauge20332);
142     ringInvList.add(gauge20532);
143     ringInvList.add(gauge20564);
144     ringInvList.add(gauge20764);
145     ringInvList.add(gauge20964);
146     ringInvList.add(gauge201164);
147 }
148 }
```



```
1  /* inventory.java
2   * Contains the inventory class functions and
   implementations
3   */
4   package c.kaitlyn.chainmaillecalculations;
5
6   import android.support.v7.app.AppCompatActivity;
7   import android.view.View;
8   import android.widget.ListView;
9
10  public class inventory extends AppCompatActivity {
11
12      private String ring_size;
13      private int count;
14
15      public inventory(){}
16
17      public inventory(String ringSize, int numCount){
18          this.count = numCount;
19          this.ring_size = ringSize;
20      }
21      public void setRing_size(String ringSize){
22          this.ring_size = ringSize;
23      }
24
25      public void setCount(int num){
26          this.count = num;
27      }
28
29      public String getRing_size(){
30          return this.ring_size;
31      }
32
33      public int getCount(){
34          return this.count;
35      }
36
37
38  }
39
```

```
1  /* ringInventory.java
2   * Contains the class information and implementation for
   the
3   * ring inventory class
4   */
5  package c.kaitlyn.chainmaillecalculations;
6
7  import android.os.Parcel;
8  import android.os.Parcelable;
9
10 public class ringInventory implements Parcelable {
11     private String ringSize;
12     private String count;
13
14     public ringInventory(String ringSize, String count) {
15         this.ringSize = ringSize;
16         this.count = count;
17     }
18
19
20     public String getRingSize() {
21         return ringSize;
22     }
23
24     public void setRingSize(String ringSize) {
25         this.ringSize = ringSize;
26     }
27
28     public String getCount() {
29         return count;
30     }
31
32     public void setCount(String count) {
33         this.count = count;
34     }
35
36
37
38     public ringInventory(Parcel in) {
39         String[] data = new String[2];
40         in.readStringArray(data);
41         this.ringSize = data[0];
42         this.count = data[1];
43     }
44
```

```
45     @Override
46     public int describeContents() {
47         return 0;
48     }
49
50     @Override
51     public void writeToParcel(Parcel dest, int flags) {
52         dest.writeStringArray(new String[]{this.ringSize,
this.count});
53     }
54
55     public static final Parcelable.Creator<ringInventory>
CREATOR = new Parcelable.Creator<ringInventory>() {
56
57         @Override
58         public ringInventory createFromParcel(Parcel
source) {
59 // TODO Auto-generated method stub
60             return new ringInventory(source); //using
parcelable constructor
61         }
62
63         @Override
64         public ringInventory[] newArray(int size) {
65 // TODO Auto-generated method stub
66             return new ringInventory[size];
67         }
68
69     };
70 }
```

```
1  /* RingInventoryListAdapter.java
2   * Connects the ringinventory and the listadapter
3   */
4  package c.kaitlyn.chainmaillecalculations;
5
6  import android.content.Context;
7  import android.support.annotation.NonNull;
8  import android.support.annotation.Nullable;
9  import android.view.LayoutInflater;
10 import android.view.View;
11 import android.view.ViewGroup;
12 import android.widget.ArrayAdapter;
13 import android.widget.ListView;
14 import android.widget.TextView;
15
16 import org.w3c.dom.Text;
17
18 import java.util.ArrayList;
19 import java.util.List;
20
21 public class RingInventoryListAdapter extends ArrayAdapter
    <ringInventory>{
22     //private static final String TAG = "
    RingInventoryListAdapter";
23
24     private Context mContext;
25     int mResource;
26
27     public RingInventoryListAdapter(@NonNull Context
    context, int resource, @NonNull ArrayList<ringInventory>
    objects) {
28         super(context, resource, objects);
29         this.mContext = context;
30         this.mResource = resource;
31     }
32
33     @NonNull
34     @Override
35     public View getView(int position, @Nullable View
    convertView, @NonNull ViewGroup parent) {
36
37         String ringSize = getItem(position).getRingSize();
38         String count = getItem(position).getCount();
39         ringInventory ringinv = new ringInventory(ringSize
    ,count);
```

```
40
41     LayoutInflater inflater = LayoutInflater.from(
mContext);
42     convertView = inflater.inflate(mResource, parent,
false);
43
44     try {
45         TextView tvRingSize = (TextView) convertView.
findViewById(R.id.ringSizeinv);
46         TextView tvCount = (TextView) convertView.
findViewById(R.id.ringInventoryCount);
47
48         tvRingSize.setText(ringSize);
49         tvCount.setText(count);
50     }catch(NullPointerException e){
51
52     }
53     return convertView;
54
55 }
56 }
57
```

```
1  /* ListViewAdapter.java
2   * Contains the information and functions of the ListView
   Adapter.
3   * This is used for the inventory pages table display.
4   */
5  package c.kaitlyn.chainmaillecalculations;
6
7  import android.app.Activity;
8  import android.view.LayoutInflater;
9  import android.view.View;
10 import android.view.ViewGroup;
11 import android.widget.BaseAdapter;
12 import android.widget.TextView;
13
14 import java.util.ArrayList;
15 import java.util.HashMap;
16
17 public class ListViewAdapter extends BaseAdapter {
18     public ArrayList<HashMap<String, String>> list;
19     Activity activity;
20     public static final String FIRST_COLUMN="Ring Size";
21     public static final String SECOND_COLUMN="Count";
22
23     public ListViewAdapter(Activity activity,ArrayList<
   HashMap<String, String>> list){
24         super();
25         this.activity=activity;
26         this.list=list;
27     }
28
29     @Override
30     public int getCount() {
31         // TODO Auto-generated method stub
32         return list.size();
33     }
34
35     @Override
36     public Object getItem(int position) {
37         // TODO Auto-generated method stub
38         return list.get(position);
39     }
40
41     @Override
42     public long getItemId(int position) {
43         // TODO Auto-generated method stub
```

```
44         return 0;
45     }
46
47     private class ViewHolder{
48         TextView txtFirst;
49         TextView txtSecond;
50     }
51
52     @Override
53     public View getView(int position, View convertView,
54     ViewGroup parent) {
55         // TODO Auto-generated method stub
56
57         ViewHolder holder;
58
59         LayoutInflater inflater=activity.getLayoutInflater
60         ();
61
62         if(convertView == null){
63
64             convertView=inflater.inflate(R.layout.
65             column_row, null);
66             holder=new ViewHolder();
67
68             holder.txtFirst=(TextView) convertView.
69             findViewById(R.id.ring_size_col);
70             holder.txtSecond=(TextView) convertView.
71             findViewById(R.id.count_col);
72
73             convertView.setTag(holder);
74         }else{
75
76             holder=(ViewHolder) convertView.getTag();
77         }
78
79         HashMap<String, String> map=list.get(position);
80         holder.txtFirst.setText(map.get(FIRST_COLUMN));
81         holder.txtSecond.setText(map.get(SECOND_COLUMN));
82
83         return convertView;
84     }
85 }
```

```
1  /* ViewPagerAdapter.java
2   * Creates the adapter for the fragments used in the
   application
3   */
4  package c.kaitlyn.chainmaillecalculations;
5
6  import android.support.annotation.Nullable;
7  import android.support.v4.app.Fragment;
8  import android.support.v4.app.FragmentManager;
9  import android.support.v4.app.FragmentPagerAdapter;
10
11 import java.util.ArrayList;
12 import java.util.List;
13
14 public class ViewPagerAdapter extends FragmentPagerAdapter
15 {
16     private final List<Fragment> fragmentList = new
        ArrayList<>();
17     private final List<String> FragmentListTitles = new
        ArrayList<>();
18
19
20     public ViewPagerAdapter(FragmentManager fm) {
21         super(fm);
22     }
23
24     @Override
25     public Fragment getItem(int position) {
26         return fragmentList.get(position);
27     }
28
29     @Override
30     public int getCount() {
31         return FragmentListTitles.size();
32     }
33
34     @Nullable
35     @Override
36     public CharSequence getPageTitle(int position) {
37         return FragmentListTitles.get(position);
38     }
39
40     public void AddFragment(Fragment fragment, String
        Title){
```



```
41         fragmentList.add(fragment);
42         FragmentListTitles.add(Title);
43     }
44 }
45
```

```
1  /* updateActivity.java
2   * Updates the page with new xml
3   */
4
5  package c.kaitlyn.chainmaillecalculations;
6
7  import android.app.Activity;
8  import android.content.Intent;
9  import android.support.v7.app.AppCompatActivity;
10 import android.os.Bundle;
11 import android.util.DisplayMetrics;
12 import android.view.View;
13 import android.widget.Button;
14 import android.widget.EditText;
15 import android.widget.TextView;
16
17 public class updateActivity extends Activity {
18
19     @Override
20     protected void onCreate(Bundle savedInstanceState) {
21         super.onCreate(savedInstanceState);
22         setContentView(R.layout.activity_update);
23
24         final ringInventory result = getIntent().
getParcelableExtra("inv");
25         DisplayMetrics dm = new DisplayMetrics();
26         getWindowManager().getDefaultDisplay().getMetrics(
dm);
27
28         int width = dm.widthPixels;
29         int height = dm.heightPixels;
30
31         getWindow().setLayout((int)(.8*width), (int)(.5*
height));
32
33
34         //get info to update
35         final String ringSize, count;
36         ringSize = result.getRingSize();
37         count = result.getCount();
38
39         TextView rs_update = findViewById(R.id.
ringSize_result);
40         TextView count_update = findViewById(R.id.
count_result);
```

```
41         final EditText editText = findViewById(R.id.
count_result);
42
43         rs_update.setText(ringSize);
44         count_update.setText(count);
45
46         Button btnUpd;
47         ringInventory  newResult;
48
49         btnUpd = this.findViewById(R.id.updateButton);
50
51         btnUpd.setOnClickListener(new View.OnClickListener
    () {
52             @Override
53             public void onClick(View v) {
54                 String countUp = editText.getText().
toString();
55                 result.setCount(countUp);
56
57             }
58         });
59         String countUpdate = result.getCount();
60         //newResult = new ringInventory(ringSize,
countUpdate);
61         Intent returnIntent = new Intent();
62         returnIntent.putExtra("result", result);
63         setResult(Activity.RESULT_OK, returnIntent);
64         finish();
65
66     }
67 }
68
```

```
1 /* spiralclass.java
2  * connects the spiral chain xml to the application
3  */
4 package c.kaitlyn.chainmaillecalculations;
5
6 import android.support.v7.app.AppCompatActivity;
7 import android.os.Bundle;
8
9 public class spiralclass extends AppCompatActivity {
10
11     @Override
12     protected void onCreate(Bundle savedInstanceState) {
13         super.onCreate(savedInstanceState);
14         setContentView(R.layout.spiralchain);
15     }
16 }
17
```

```
1  /* shaggyclass.java
2   * connects the shaggy loops xml to the application
3   */
4  package c.kaitlyn.chainmaillecalculations;
5
6  import android.support.v7.app.AppCompatActivity;
7  import android.os.Bundle;
8
9  public class shaggyclass extends AppCompatActivity {
10
11      @Override
12      protected void onCreate(Bundle savedInstanceState) {
13          super.onCreate(savedInstanceState);
14          setContentView(R.layout.shaggychain);
15      }
16  }
17
```

```
1  /* boxchain.java
2   * connects the box chain xml to the application
3   */
4  package c.kaitlyn.chainmaillecalculations;
5
6  import android.support.v7.app.AppCompatActivity;
7  import android.os.Bundle;
8
9  public class boxchain extends AppCompatActivity {
10
11      @Override
12      protected void onCreate(Bundle savedInstanceState) {
13          super.onCreate(savedInstanceState);
14          setContentView(R.layout.boxchain);
15      }
16  }
17
```

```
1  /* inventory_tab.java
2   * Contains all of the java for the inventory tab of the
   application.
3   * Uses shared preferences to store the inventory data.
4   * Uses an xml file as the layout for the fragment.
5   */
6  package c.kaitlyn.chainmaillecalculations.fragments;
7
8  import android.app.Dialog;
9  import android.content.Context;
10 import android.content.Intent;
11 import android.content.SharedPreferences;
12 import android.support.annotation.NonNull;
13 import android.support.v4.app.Fragment;
14 import android.os.Bundle;
15 import android.support.annotation.Nullable;
16 import android.util.Log;
17 import android.view.LayoutInflater;
18 import android.view.View;
19 import android.view.ViewGroup;
20 import android.widget.AdapterView;
21 import android.widget.Button;
22 import android.widget.EditText;
23 import android.widget.ListView;
24 import android.widget.TextView;
25 import com.google.gson.Gson;
26 import com.google.gson.reflect.TypeToken;
27 import java.lang.reflect.Type;
28 import java.util.ArrayList;
29 import c.kaitlyn.chainmaillecalculations.R;
30 import c.kaitlyn.chainmaillecalculations.
    RingInventoryListAdapter;
31 import c.kaitlyn.chainmaillecalculations.ringInventory;
32
33 public class inventory_tab extends Fragment {
34
35     View view;
36     private static final String TAG = "inventory";
37
38     ArrayList<ringInventory> ringInvList;
39
40
41     @Override
42     public void setUserVisibleHint(boolean isVisibleToUser
    ) {
```

```
43         super.setUserVisibleHint(isVisibleToUser);
44         if (isVisibleToUser) {
45             getFragmentManager().beginTransaction().detach
(this).attach(this).commit();
46         }
47     }
48
49     @Nullable
50     @Override
51     public View onCreateView(@NonNull LayoutInflater
inflater, @Nullable ViewGroup container, @Nullable Bundle
savedInstanceState) {
52         view = inflater.inflate(R.layout.inventory_tab,
container, false);
53
54         //ringInvList.clear();
55         loadData();
56
57         final ListView mListView = (ListView) view.
findViewById(R.id.RingInventory);
58         final RingInventoryListAdapter adapter = new
RingInventoryListAdapter(getActivity(), R.layout.
adapter_view_layout, ringInvList);
59         mListView.setAdapter(adapter);
60
61         mListView.setOnItemClickListener(new AdapterView.
OnItemClickListener() {
62             @Override
63             public void onItemClick(AdapterView<?> parent,
View view, final int position, long id) {
64                 loadData();
65                 ringInventory upd = ringInvList.get(position);
66                 final Dialog dialog=new Dialog(getContext());
67                 dialog.setTitle("Update inventory");
68                 dialog.setContentView(R.layout.
activity_update);
69                 final String rs = upd.getRingSize();
70                 String count = upd.getCount();
71                 TextView txtRS = dialog.findViewById(R.id.
ringSize_result);
72                 txtRS.setText(rs);
73                 final EditText txtCount = dialog.findViewById
(R.id.count_result);
74                 txtCount.setText(count);
75
```



```

76         Button bt = dialog.findViewById(R.id.
updateButton);
77         bt.setOnClickListener(new View.
OnClickListener() {
78             @Override
79             public void onClick(View v) {
80                 final ringInventory up = new
ringInventory(rs, txtCount.getText().toString());
81                 ringInvList.set(position, up);
82                 ringInventory test = ringInvList.
get(position);
83                 Log.d(TAG, "STATE: up = " + up.
getCount());
84                 Log.d("STATE", test.getCount());
85                 saveData();
86                 adapter.notifyDataSetChanged();
87                 dialog.dismiss();
88                 //loadData();
89             }
90
91         });
92         loadData();
93         dialog.show();
94
95     }
96
97     });
98     return view;
99 }
100
101
102     private void editList(final ListView listView){
103         listView.setOnItemClickListener(new AdapterView.
OnItemClickListener() {
104             @Override
105             public void onItemClick(AdapterView<?>
parent, View view, int position, long id) {
106                 editBox(ringInvList.get(position),
position);
107             }
108
109         });
110
111     }
112

```

```
113         public void editBox(ringInventory inventory,
114             final int pos){
115             final Dialog dialog=new Dialog(getContext());
116             dialog.setTitle("Update inventory");
117             dialog.setContentView(R.layout.
118                 activity_update);
119             final String rs = inventory.getRingSize();
120             String count = inventory.getCount();
121             TextView txtRS = dialog.findViewById(R.id.
122                 ringSize_result);
123             txtRS.setText(rs);
124             EditText txtCount = dialog.findViewById(R.id.
125                 count_result);
126             txtCount.setText(count);
127             final ringInventory up = new ringInventory(rs
128                 , txtCount.getText().toString());
129             Button bt = dialog.findViewById(R.id.
130                 updateButton);
131             bt.setOnClickListener(new View.
132                 OnClickListener() {
133                 @Override
134                 public void onClick(View v) {
135                     ringInvList.set(pos, up);
136                     dialog.dismiss();
137                 }
138             });
139             dialog.show();
140         }
141         @Override
142         public void onActivityResult(int requestCode, int
143             resultCode, Intent data) {
144             super.onActivityResult(requestCode,
145                 resultCode, data);
146         }
147         private void saveData(){
148             SharedPreferences sharedPreferences = getContext().
149                 getSharedPreferences("shared preferences", Context.
150                     MODE_PRIVATE);
151             SharedPreferences.Editor editor = sharedPreferences.
152                 edit();
153             Gson gson = new Gson();
154             String json = gson.toJson(ringInvList);
155             Log.d(TAG,"STATE: save json = "+json);
```

```
146         editor.putString("inventory", json);
147         editor.apply();
148
149     }
150
151     private void loadData(){
152         SharedPreferences sharedPreferences = getContext(
153         ).getSharedPreferences("shared preferences", Context.
154         MODE_PRIVATE);
155
156         Gson gson = new Gson();
157         String json = sharedPreferences.getString("
158         inventory", null);
159
160         //sharedPreferences.edit().clear().commit();
161
162         Type type = new TypeToken<ArrayList<ringInventory
163         >>() {}.getType();
164
165         ringInvList = gson.fromJson(json, type);
166
167         if (ringInvList == null){
168             ringInvList = new ArrayList<>();
169             populateList();
170         }
171     }
172
173     private void populateList() {
174
175         ringInventory gauge1614 = new ringInventory("16g 1/
176         4in", "2400");
177         ringInventory gauge1638 = new ringInventory("16g 3/
178         8in", "650");
179         ringInventory gauge16516 = new ringInventory("16g 5/
180         16in", "750");
181         ringInventory gauge16716 = new ringInventory("16g 7/
182         16in", "650");
183         ringInventory gauge16732 = new ringInventory("16g 7/
184         32in", "300");
185         ringInventory gauge1814 = new ringInventory("18g 1/
186         4in", "200");
187         ringInventory gauge1818 = new ringInventory("18g 1/
188         8in", "600");
189         ringInventory gauge18316 = new ringInventory("18g 3/
190         16in", "5000");
191         ringInventory gauge18516 = new ringInventory("18g 5/
```

```
178 16in", "1000");
179     ringInventory gauge18532 = new ringInventory("18g 5/
32in", "500");
180     ringInventory gauge18732 = new ringInventory("18g 7/
32in", "700");
181     ringInventory gauge1914 = new ringInventory("19g 1/
4in", "200");
182     ringInventory gauge19316 = new ringInventory("19g 3/
16in", "100");
183     ringInventory gauge19532 = new ringInventory("19g 5/
32in", "300");
184     ringInventory gauge19964 = new ringInventory("19g 9/
64in", "100");
185     ringInventory gauge2018 = new ringInventory("20g 1/
8in", "2100");
186     ringInventory gauge20316 = new ringInventory("20g 3/
16in", "2400");
187     ringInventory gauge20332 = new ringInventory("20g 3/
32in", "3950");
188     ringInventory gauge20532 = new ringInventory("20g 5/
32in", "930");
189     ringInventory gauge20564 = new ringInventory("20g 5/
64in", "300");
190     ringInventory gauge20764 = new ringInventory("20g 7/
64in", "1600");
191     ringInventory gauge20964 = new ringInventory("20g 9/
64in", "2950");
192     ringInventory gauge201164 = new ringInventory("20g
11/64in", "2400");
193
194
195
196     ringInvList.add(gauge1614);
197     ringInvList.add(gauge1638);
198     ringInvList.add(gauge16516);
199     ringInvList.add(gauge16716);
200     ringInvList.add(gauge16732);
201     ringInvList.add(gauge1814);
202     ringInvList.add(gauge1818);
203     ringInvList.add(gauge18316);
204     ringInvList.add(gauge18516);
205     ringInvList.add(gauge18532);
206     ringInvList.add(gauge18732);
207     ringInvList.add(gauge1914);
208     ringInvList.add(gauge19316);
```

```
209     ringInvList.add(gauge19532);
210     ringInvList.add(gauge19964);
211     ringInvList.add(gauge2018);
212     ringInvList.add(gauge20316);
213     ringInvList.add(gauge20332);
214     ringInvList.add(gauge20532);
215     ringInvList.add(gauge20564);
216     ringInvList.add(gauge20764);
217     ringInvList.add(gauge20964);
218     ringInvList.add(gauge201164);
219 }
220 }
221
```

```
1  /* instruction_tab.java
2  * Contains all of the java for the instruction tab of the
   application.
3  * Uses an xml file as the layout for the fragment.
4  * Contains buttons that display instructions with steps
   and images.
5  */
6  package c.kaitlyn.chainmaillecalculations.fragments;
7
8  import android.content.Intent;
9  import android.support.annotation.NonNull;
10 import android.support.v4.app.Fragment;
11 import android.os.Bundle;
12 import android.support.annotation.Nullable;
13 import android.support.v4.app.Fragment;
14 import android.view.LayoutInflater;
15 import android.view.View;
16 import android.view.ViewGroup;
17 import android.widget.Button;
18
19 import c.kaitlyn.chainmaillecalculations.R;
20 import c.kaitlyn.chainmaillecalculations.boxchain;
21 import c.kaitlyn.chainmaillecalculations.shaggyclass;
22 import c.kaitlyn.chainmaillecalculations.spiralclass;
23
24 public class instruction_tab extends Fragment {
25     View view;
26
27     //Creates buttons for page use
28     private Button box, spiral, shaggy;
29     public instruction_tab() {
30     }
31
32     /*
33         Creates the fragment view that the user interacts
   with
34     */
35     @Nullable
36     @Override
37     public View onCreateView(@NonNull LayoutInflater
   inflater, @Nullable ViewGroup container, @Nullable Bundle
   savedInstanceState) {
38         view = inflater.inflate(R.layout.instruction_tab,
   container, false);
39         box = view.findViewById(R.id.boxbutton);
```

```
40         spiral = view.findViewById(R.id.spiralbutton);
41         shaggy = view.findViewById(R.id.shaggybutton);
42
43         box.setOnClickListener(new View.OnClickListener() {
44
45             @Override
46             public void onClick(View v) {
47                 openBoxinst();
48             }
49         });
50
51         spiral.setOnClickListener(new View.OnClickListener
52         () {
53
54             @Override
55             public void onClick(View v) {
56                 openSpiralinst();
57             }
58         });
59         shaggy.setOnClickListener(new View.OnClickListener
60         () {
61
62             @Override
63             public void onClick(View v) {
64                 openShaggyinst();
65             }
66         });
67         return view;
68
69     public void openBoxinst(){
70         Intent intent = new Intent(getApplicationContext(), boxchain.
71         class);
72         startActivity(intent);
73     }
74     public void openSpiralinst(){
75         Intent intent2 = new Intent(getApplicationContext(),
76         spiralclass.class);
77         startActivity(intent2);
78     }
79     public void openShaggyinst(){
80         Intent intent3 = new Intent(getApplicationContext(),
81         shaggyclass.class);
82         startActivity(intent3);
83     }
84 }
```

80 }

81 }

82


```
1  /* calc_tab.java
2   * Contains all of the java for the calculations tab of
   the application.
3   * Uses an xml file as the layout for the fragment.
4   */
5
6  package c.kaitlyn.chainmaillecalculations.fragments;
7
8  import android.app.AlertDialog;
9  import android.app.Dialog;
10 import android.content.Context;
11 import android.content.SharedPreferences;
12 import android.graphics.Color;
13 import android.support.annotation.NonNull;
14 import android.support.v4.app.DialogFragment;
15 import android.support.v4.app.Fragment;
16 import android.os.Bundle;
17 import android.support.annotation.Nullable;
18 import android.support.v4.app.Fragment;
19 import android.util.Log;
20 import android.view.LayoutInflater;
21 import android.view.View;
22 import android.view.ViewGroup;
23 import android.widget.AdapterView;
24 import android.widget.ArrayAdapter;
25 import android.widget.Button;
26 import android.widget.EditText;
27 import android.widget.Spinner;
28 import android.widget.TextView;
29
30 import com.google.gson.Gson;
31 import com.google.gson.reflect.TypeToken;
32
33 import java.lang.reflect.Type;
34 import java.util.ArrayList;
35
36 import c.kaitlyn.chainmaillecalculations.R;
37 import c.kaitlyn.chainmaillecalculations.ringInventory;
38
39 import static android.content.ContentValues.TAG;
40
41 public class calc_tab extends Fragment {
42     View view;
43     //Code for calculate page, sets up variables for page
44     private TextView mTextMessage;
```

```
45
46     double length;
47     int open, closed;
48     String gauge, ringSize, ringSizeStr, weave, errortight
    = "Too tight, choose another size";
49     String errorloose = "Too loose, choose another size";
50     boolean isTight = false, isLoose = false;
51     String dialog_title = "Error - improper size";
52     String dialog_message = "Choose a different ring size
    , this size is not appropriate for this weave";
53
54     ArrayList<ringInventory> ringInvList;
55
56     public calc_tab() {
57
58     }
59
60     //OncreateView sets up the view fragment that is used
    as a tab on the application
61     @Nullable
62     @Override
63     public View onCreateView(@NonNull LayoutInflater
    inflater, @Nullable ViewGroup container, @Nullable Bundle
    savedInstanceState) {
64         //Sets the view
65         view = inflater.inflate(R.layout.calc_tab,
    container, false);
66         //Calls the function that Loads the data from
    shared preferences
67         loadData();
68         //Code for calculate
69         //Weave types
70         Spinner weavespinner = view.findViewById(R.id.
    weavenames);
71
72         ArrayAdapter<CharSequence> adapter = ArrayAdapter.
    createFromResource(getContext(),
73             R.array.weaves, android.R.layout.
    simple_spinner_item);
74
75         // Specify the layout to use when the list of
    choices appears
76         adapter.setDropDownViewResource(android.R.layout.
    simple_spinner_dropdown_item);
77
```

```
78         // Apply the adapter to the spinner
79         weavespinner.setAdapter(adapter);
80         weavespinner.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
81             //When an item from the Weaves Dropdown is
selected the following code runs
82             @Override
83             public void onItemSelected(AdapterView<?>
parent, View view, int position, long id) {
84                 String selectedweave = parent.
getItemAtPosition(position).toString();
85
86                 switch(selectedweave){
87                     case "Spiral":
88                         weave = "Spiral";
89                         break;
90                     case "Box":
91                         weave = "Box";
92                         break;
93                     case "Shaggy Loops":
94                         weave = "Shaggy Loops";
95                         break;
96                 }
97             }
98
99             @Override
100             public void onNothingSelected(AdapterView<?>
parent) {
101
102             }
103         });
104
105
106         //Ring Gauge Size
107         Spinner gaugespinner = view.findViewById(R.id.
ring_gaugearray);
108
109         //Ring Size(inch)
110         final Spinner sizespinner = view.findViewById(R.
id.ringSizespinner);
111
112         //when gauge dropdown item selected
113         gaugespinner.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
114             @Override
```

```
115         public void onItemSelected(AdapterView<?>
        parent, View view, int position, long id) {
116             //makes size spinner visible and have the
            correct sizes
117
118             String selectedGauge = parent.
getItemAtPosition(position).toString();
119             switch (selectedGauge){
120                 /* Create an ArrayAdapter using the
string array and a default spinner layout
121                 from the selected gauge size
122                 */
123                 case "16":
124                     sizespinner.setAdapter(new
ArrayAdapter<String>(getActivity(),
125                     android.R.layout.
simple_spinner_dropdown_item,
126                     getResources().
getStringArray(R.array.ring_gauge16)));
127
128                     gauge = "16";
129                     break;
130
131                 case "18":
132                     sizespinner.setAdapter(new
ArrayAdapter<String>(getActivity(),
133                     android.R.layout.
simple_spinner_dropdown_item,
134                     getResources().
getStringArray(R.array.ring_gauge18)));
135                     gauge = "18";
136                     break;
137
138                 case "19":
139                     sizespinner.setAdapter(new
ArrayAdapter<String>(getActivity(),
140                     android.R.layout.
simple_spinner_dropdown_item,
141                     getResources().
getStringArray(R.array.ring_gauge19)));
142                     gauge = "19";
143                     break;
144
145                 case "20":
146                     sizespinner.setAdapter(new
```

```

146 ArrayAdapter<String>(getActivity(),
147                        android.R.layout.
    simple_spinner_dropdown_item,
148                        getResources().
    getStringArray(R.array.ring_gauge20)));
149                        gauge = "20";
150                        break;
151                }
152        }
153
154        @Override
155        public void onNothingSelected(AdapterView<?>
parent) {
156                //nothing happens if nothing is selected
157                try {
158                        TextView ringSizetext = view.
    findViewById(R.id.ring_size);
159                        Spinner ringsizespinner = view.
    findViewById(R.id.ringSizespinner);
160
161                        ringSizetext.setVisibility(View.GONE)
;
162                        ringsizespinner.setVisibility(View.
    GONE);
163                }catch(NullPointerException e){
164
165                }
166        }
167    });
168
169    //Sets the ring size variables for use from the
    selected value in the dropdown
170    sizespinner.setOnItemClickListener(new
    AdapterView.OnItemClickListener() {
171        @Override
172        public void onItemClick(AdapterView<?>
parent, View view, int position, long id) {
173                String selectedSize = parent.
    getItemAtPosition(position).toString();
174                switch(selectedSize) {
175                    case "1/4":
176                        ringSize = "1/4";
177                        ringSizeStr = "14";
178                        break;
179                    case "1/8":

```

```
180         ringSize = "1/8";
181         ringSizeStr = "18";
182         break;
183     case "3/8":
184         ringSize = "3/8";
185         ringSizeStr = "38";
186         break;
187     case "3/16":
188         ringSize = "3/16";
189         ringSizeStr = "316";
190         break;
191     case "5/16":
192         ringSize = "5/16";
193         ringSizeStr = "516";
194         break;
195     case "7/16":
196         ringSize = "7/16";
197         ringSizeStr = "716";
198         break;
199     case "5/32":
200         ringSize = "5/32";
201         ringSizeStr = "532";
202         break;
203     case "7/32":
204         ringSize = "7/32";
205         ringSizeStr = "732";
206         break;
207     case "3/32":
208         ringSize = "3/32";
209         ringSizeStr = "332";
210         break;
211     case "5/64":
212         ringSize = "5/64";
213         ringSizeStr = "564";
214         break;
215     case "7/64":
216         ringSize = "7/64";
217         ringSizeStr = "764";
218         break;
219     case "9/64":
220         ringSize = "9/64";
221         ringSizeStr = "964";
222         break;
223     case "11/64":
224         ringSize = "11/64";
```



```
260         case "6.5":
261             length = 6.5;
262             break;
263         case "7":
264             length = 7;
265             break;
266         case "8":
267             length = 8;
268             break;
269         case "7.5":
270             length = 7.5;
271             break;
272         case "8.5":
273             length = 8.5;
274             break;
275         case "24":
276             length = 24;
277             break;
278         case "28":
279             length = 28;
280             break;
281     }
282 }
283 @Override
284 public void onNothingSelected(AdapterView<?>
parent) {}
285     });
286
287     /*
288         Calculate button is created so that the user
289         can press it and the open ring and the
290         Closed rings will be calculated and displayed
291         */
292     final Button calbutton = view.findViewById(R.id.
calculatebutton);
293     calbutton.setOnClickListener(new View.
OnClickListener() {
294         public void onClick(View v) {
295             // Code here executes on main thread
296             after user presses button
297             calculateTotals(weave, gauge, ringSize,
length);
298         }
299     });
300 }
```



```
299         /*
300             Button is created that subtracts from the
inventory and updates the shared
301             preferences
302         */
303         final Button subbutton = view.findViewById(R.id.
subtractButton);
304         subbutton.setOnClickListener(new View.
OnClickListener() {
305             public void onClick(View v) {
306                 loadData();
307                 int total = open + closed;
308
309                 String ring = gauge+"g "+ ringSize + "in"
;
310                 String tempSize="", tempCount="";
311                 Integer i=0,pos=0;
312
313                 for(RingInventory ringInv : ringInvList){
314                     if(ringInv.getRingSize().equals(ring)
){
315                         ringInventory temp = ringInv;
316                         tempSize = ringInv.getRingSize();
317                         tempCount = ringInv.getCount();
318                         Log.d(TAG, "tempVal: ring count
of "+tempSize+ " is " + tempCount);
319                         pos = i;
320                     }
321                     else{
322                         Log.d(TAG, "tempVal: Can't find")
;
323                     }
324                     i++;
325
326                 }
327
328                 Integer sub = Integer.parseInt(tempCount)
;
329                 Integer fin = sub - total;
330                 ringInventory tempRing = new
ringInventory(tempSize,Integer.toString(fin));
331                 ringInvList.set(pos,tempRing);
332                 Log.d(TAG, "SETNEW: total to subtract"+
Integer.toString(total));
333                 Log.d(TAG, "SETNEW: setting the new ring
```

```
333     " + tempRing.getRingSize()+ " to " + Integer.toString(fin
    ));
334
335
336         AlertDialog.Builder builder = new
AlertDialog.Builder(getActivity());
337         builder.setMessage("Updating the value
for "+tempSize +" to " + fin);
338         builder.setTitle("Update Value");
339
340         saveData();
341     }
342     });
343     return view;
344 }
345
346     /*
347         Saves all of the data to the shared preferences
to be saved as a "Database"
348     */
349     private void saveData(){
350         SharedPreferences sharedPreferences = getContext(
).getSharedPreferences("shared preferences", Context.
MODE_PRIVATE);
351         SharedPreferences.Editor editor =
sharedPreferences.edit();
352         Gson gson = new Gson();
353         String json = gson.toJson(ringInvList);
354         editor.putString("inventory", json);
355         editor.apply();
356     }
357
358     /*
359         Loads all of the data from the shared preferences
to be used in the calculation tab
360     */
361     private void loadData(){
362         SharedPreferences sharedPreferences = getContext(
).getSharedPreferences("shared preferences", Context.
MODE_PRIVATE);
363         Gson gson = new Gson();
364         String json = sharedPreferences.getString("
inventory", null);
365
366         Type type = new TypeToken<ArrayList<ringInventory
```

```
366 >>() {}.getType();
367         ringInvList = gson.fromJson(json, type);
368
369         if (ringInvList == null){
370             ringInvList = new ArrayList<>();
371         }
372     }
373     public int calculateTotals(String weave, String gauge
, String ringSize, double length){
374         if (weave.equals("Spiral")) {
375             if (gauge.equals("16")) {
376                 if (ringSize.equals("1/4")) {
377                     open = (int) (length * 7);
378                     closed = (int) (length * 0);
379                     isTight = false;
380                     isLoose = false;
381                 }
382                 if (ringSize.equals("3/8")) {
383                     open = (int) (length * 4);
384                     closed = (int) (length * 0);
385                     isTight = false;
386                     isLoose = false;
387                 }
388                 if (ringSize.equals("5/16")) {
389                     open = (int) (length * 5);
390                     closed = (int) (length * 0);
391                     isTight = false;
392                     isLoose = false;
393                 }
394                 if (ringSize.equals("7/16")) {
395                     open = (int) (length * 3);
396                     closed = (int) (length * 0);
397                     isTight = false;
398                     isLoose = false;
399                 }
400                 if (ringSize.equals("7/32")) {
401                     open = (int) (length * 0);
402                     closed = (int) (length * 0);
403                     isTight = true;
404                     isLoose = false;
405                 }
406             }
407             if (gauge.equals("18")) {
408                 if (ringSize.equals("1/4")) {
409                     open = (int) (length * 6);
```

```
410         closed = (int) (length * 0);
411         isTight = false;
412         isLoose = false;
413     }
414     if (ringSize.equals("5/16")) {
415         open = (int) (length * 6);
416         closed = (int) (length * 0);
417         isTight = false;
418         isLoose = false;
419     }
420     if (ringSize.equals("7/32")) {
421         open = (int) (length * 7);
422         closed = (int) (length * 0);
423         isTight = false;
424         isLoose = false;
425     }
426     if (ringSize.equals("1/8")) {
427         open = (int) (length * 0);
428         closed = (int) (length * 0);
429         isTight = true;
430         isLoose = false;
431     }
432     if (ringSize.equals("3/16")) {
433         open = (int) (length * 9);
434         closed = (int) (length * 0);
435         isTight = false;
436         isLoose = false;
437     }
438     if (ringSize.equals("5/32")) {
439         open = (int) (length * 0);
440         closed = (int) (length * 0);
441         isTight = true;
442         isLoose = false;
443     }
444
445 }
446 if (gauge.equals("19")) {
447     if (ringSize.equals("1/4")) {
448         open = (int) (length * 7);
449         closed = (int) (length * 0);
450         isTight = false;
451         isLoose = false;
452     }
453     if (ringSize.equals("3/16")) {
454         open = (int) (length * 8);
```

```
455         closed = (int) (length * 0);
456         isTight = false;
457         isLoose = false;
458     }
459     if (ringSize.equals("5/32")) {
460         open = (int) (length * 10);
461         closed = (int) (length * 0);
462         isTight = false;
463         isLoose = false;
464     }
465     if (ringSize.equals("9/64")) {
466         open = (int) (length * 12);
467         closed = (int) (length * 0);
468         isTight = false;
469         isLoose = false;
470     }
471 }
472 if (gauge.equals("20")) {
473
474     if (ringSize.equals("1/8")) {
475         open = (int) (length * 15);
476         closed = (int) (length * 0);
477         isTight = false;
478         isLoose = false;
479     }
480     if (ringSize.equals("3/16")) {
481         open = (int) (length * 9);
482         closed = (int) (length * 0);
483         isTight = false;
484         isLoose = false;
485     }
486     if (ringSize.equals("5/32")) {
487         open = (int) (length * 12);
488         closed = (int) (length * 0);
489         isTight = false;
490         isLoose = false;
491     }
492     if (ringSize.equals("9/64")) {
493         open = (int) (length * 13);
494         closed = (int) (length * 0);
495         isTight = false;
496         isLoose = false;
497     }
498     if (ringSize.equals("3/32")) {
499         open = (int) (length * 0);
```

```
500         closed = (int) (length * 0);
501         isTight = true;
502         isLoose = false;
503     }
504     if (ringSize.equals("5/64")) {
505         open = (int) (length * 0);
506         closed = (int) (length * 0);
507         isTight = true;
508         isLoose = false;
509     }
510     if (ringSize.equals("7/64")) {
511         open = (int) (length * 0);
512         closed = (int) (length * 0);
513         isTight = true;
514         isLoose = false;
515     }
516     if (ringSize.equals("11/64")) {
517         open = (int) (length * 9);
518         closed = (int) (length * 0);
519         isTight = false;
520         isLoose = false;
521     }
522 }
523 }
524 if (weave == "Box") {
525     if (gauge.equals("16")) {
526         if (ringSize.equals("1/4")) {
527             open = (int) (length * 10);
528             closed = (int) (length * 12);
529             isTight = false;
530             isLoose = false;
531         }
532         if (ringSize.equals("3/8")) {
533             open = (int) (length * 6);
534             closed = (int) (length * 8);
535             isTight = false;
536             isLoose = true;
537         }
538         if (ringSize.equals("5/16")) {
539             open = (int) (length * 6);
540             closed = (int) (length * 8);
541             isTight = false;
542             isLoose = false;
543         }
544         if (ringSize.equals("7/16")) {
```

```
545         open = (int) (length * 4);
546         closed = (int) (length * 6);
547         isTight = false;
548         isLoose = true;
549     }
550     if (ringSize.equals("7/32")) {
551         open = (int) (length * 0);
552         closed = (int) (length * 0);
553         isTight = true;
554         isLoose = false;
555     }
556 }
557 if (gauge.equals("18")) {
558     if (ringSize.equals("1/4")) {
559         open = (int) (length * 8);
560         closed = (int) (length * 10);
561         isTight = false;
562         isLoose = false;
563     }
564     if (ringSize.equals("5/16")) {
565         open = (int) (length * 6);
566         closed = (int) (length * 8);
567         isTight = false;
568         isLoose = true;
569     }
570     if (ringSize.equals("7/32")) {
571         open = (int) (length * 12);
572         closed = (int) (length * 14);
573         isTight = false;
574         isLoose = false;
575     }
576     if (ringSize.equals("1/8")) {
577         open = (int) (length * 0);
578         closed = (int) (length * 0);
579         isTight = true;
580         isLoose = false;
581     }
582     if (ringSize.equals("3/16")) {
583         open = (int) (length * 14);
584         closed = (int) (length * 16);
585         isTight = false;
586         isLoose = false;
587     }
588     if (ringSize.equals("5/32")) {
589         open = (int) (length * 0);
```

```
590         closed = (int) (length * 0);
591         isTight = true;
592         isLoose = false;
593     }
594
595 }
596 if (gauge.equals("19")) {
597     if (ringSize.equals("1/4")) {
598         open = (int) (length * 8);
599         closed = (int) (length * 10);
600         isTight = false;
601         isLoose = true;
602     }
603     if (ringSize.equals("3/16")) {
604         open = (int) (length * 12);
605         closed = (int) (length * 14);
606         isTight = false;
607         isLoose = false;
608     }
609     if (ringSize.equals("5/32")) {
610         open = (int) (length * 16);
611         closed = (int) (length * 18);
612         isTight = false;
613         isLoose = false;
614     }
615     if (ringSize.equals("9/64")) {
616         open = (int) (length * 14);
617         closed = (int) (length * 16);
618         isTight = false;
619         isLoose = false;
620     }
621 }
622 if (gauge.equals("20")) {
623
624     if (ringSize.equals("1/8")) {
625         open = (int) (length * 24);
626         closed = (int) (length * 26);
627         isTight = false;
628         isLoose = false;
629     }
630     if (ringSize.equals("3/16")) {
631         open = (int) (length * 14);
632         closed = (int) (length * 16);
633         isTight = false;
634         isLoose = false;
```



```
635         }
636         if (ringSize.equals("5/32")) {
637             open = (int) (length * 0);
638             closed = (int) (length * 0);
639             isTight = true;
640             isLoose = false;
641         }
642         if (ringSize.equals("9/64")) {
643             open = (int) (length * 0);
644             closed = (int) (length * 0);
645             isTight = true;
646             isLoose = false;
647         }
648         if (ringSize.equals("3/32")) {
649             open = (int) (length * 0);
650             closed = (int) (length * 0);
651             isTight = true;
652             isLoose = false;
653         }
654         if (ringSize.equals("5/64")) {
655             open = (int) (length * 0);
656             closed = (int) (length * 0);
657             isTight = true;
658             isLoose = false;
659         }
660         if (ringSize.equals("7/64")) {
661             open = (int) (length * 0);
662             closed = (int) (length * 0);
663             isTight = true;
664             isLoose = false;
665         }
666         if (ringSize.equals("11/64")) {
667             open = (int) (length * 10);
668             closed = (int) (length * 12);
669             isTight = false;
670             isLoose = true;
671         }
672     }
673 }
674 if (weave == "Shaggy Loops") {
675     if (gauge.equals("16")) {
676         if (ringSize.equals("1/4")) {
677             open = (int) (length * 4);
678             closed = (int) (length * 8);
679             isTight = false;
```

```
680         isLoose = false;
681     }
682     if (ringSize.equals("3/8")) {
683         open = (int) (length * 3);
684         closed = (int) (length * 6);
685         isTight = false;
686         isLoose = false;
687     }
688     if (ringSize.equals("5/16")) {
689         open = (int) (length * 3);
690         closed = (int) (length * 6);
691         isTight = false;
692         isLoose = false;
693     }
694     if (ringSize.equals("7/16")) {
695         open = (int) (length * 2);
696         closed = (int) (length * 4);
697         isTight = false;
698         isLoose = false;
699     }
700     if (ringSize.equals("7/32")) {
701         open = (int) (length * 4);
702         closed = (int) (length * 8);
703         isTight = false;
704         isLoose = false;
705     }
706 }
707 if (gauge.equals("18")) {
708     if (ringSize.equals("1/4")) {
709         open = (int) (length * 4);
710         closed = (int) (length * 8);
711         isTight = false;
712         isLoose = false;
713     }
714     if (ringSize.equals("5/16")) {
715         open = (int) (length * 3);
716         closed = (int) (length * 6);
717         isTight = false;
718         isLoose = true;
719     }
720     if (ringSize.equals("7/32")) {
721         open = (int) (length * 4);
722         closed = (int) (length * 8);
723         isTight = false;
724         isLoose = false;
```

```
725         }
726         if (ringSize.equals("1/8")) {
727             open = (int) (length * 0);
728             closed = (int) (length * 0);
729             isTight = true;
730             isLoose = false;
731         }
732         if (ringSize.equals("3/16")) {
733             open = (int) (length * 5);
734             closed = (int) (length * 10);
735             isTight = false;
736             isLoose = false;
737         }
738         if (ringSize.equals("5/32")) {
739             open = (int) (length * 6);
740             closed = (int) (length * 12);
741             isTight = false;
742             isLoose = false;
743         }
744     }
745 }
746 if (gauge.equals("19")) {
747     if (ringSize.equals("1/4")) {
748         open = (int) (length * 4);
749         closed = (int) (length * 8);
750         isTight = false;
751         isLoose = false;
752     }
753     if (ringSize.equals("3/16")) {
754         open = (int) (length * 5);
755         closed = (int) (length * 10);
756         isTight = false;
757         isLoose = false;
758     }
759     if (ringSize.equals("5/32")) {
760         open = (int) (length * 6);
761         closed = (int) (length * 12);
762         isTight = false;
763         isLoose = false;
764     }
765     if (ringSize.equals("9/64")) {
766         open = (int) (length * 6);
767         closed = (int) (length * 12);
768         isTight = false;
769         isLoose = false;
```

```
770         }
771     }
772     if (gauge.equals("20")) {
773
774         if (ringSize.equals("1/8")) {
775             open = (int) (length * 8);
776             closed = (int) (length * 16);
777             isTight = false;
778             isLoose = false;
779         }
780         if (ringSize.equals("3/16")) {
781             open = (int) (length * 5);
782             closed = (int) (length * 10);
783             isTight = false;
784             isLoose = false;
785         }
786         if (ringSize.equals("5/32")) {
787             open = (int) (length * 6);
788             closed = (int) (length * 12);
789             isTight = false;
790             isLoose = false;
791         }
792         if (ringSize.equals("9/64")) {
793             open = (int) (length * 7);
794             closed = (int) (length * 14);
795             isTight = false;
796             isLoose = false;
797         }
798         if (ringSize.equals("3/32")) {
799             open = (int) (length * 0);
800             closed = (int) (length * 0);
801             isTight = true;
802             isLoose = false;
803         }
804         if (ringSize.equals("5/64")) {
805             open = (int) (length * 0);
806             closed = (int) (length * 0);
807             isTight = true;
808             isLoose = false;
809         }
810         if (ringSize.equals("7/64")) {
811             open = (int) (length * 8);
812             closed = (int) (length * 16);
813             isTight = false;
814             isLoose = false;
```

```
815         }
816         if (ringSize.equals("11/64")) {
817             open = (int) (length * 5);
818             closed = (int) (length * 10);
819             isTight = false;
820             isLoose = false;
821         }
822     }
823 }
824
825     try{
826         if (!isTight && !isLoose) {
827             TextView openText = view.findViewById(R.id.
openCal);
828             openText.setText(" " + Integer.toString(open)
+ " rings");
829             openText.setTextColor(Color.BLACK);
830             TextView closedText = view.findViewById(R.id.
closedCal);
831             closedText.setText(" " + Integer.toString(
closed) + " rings");
832             closedText.setTextColor(Color.BLACK);
833
834             } else if (isTight) {
835                 TextView openText = view.findViewById(R.id.
openCal);
836                 openText.setText(" " + errortight);
837                 openText.setTextColor(Color.RED);
838                 TextView closedText = view.findViewById(R.id.
closedCal);
839                 closedText.setText(" " + errortight);
840                 closedText.setTextColor(Color.RED);
841             } else if (isLoose){
842                 TextView openText = view.findViewById(R.id.
openCal);
843                 openText.setText(" " + errorloose);
844                 openText.setTextColor(Color.RED);
845                 TextView closedText = view.findViewById(R.id.
closedCal);
846                 closedText.setText(" " + errorloose);
847                 closedText.setTextColor(Color.RED);
848             }}
849         catch (NullPointerException e){
850
851         }
```

```
852         return open + closed;
853
854     }
855
856     @Override
857     public void onCreate(@Nullable Bundle
        savedInstanceState) {
858         super.onCreate(savedInstanceState);
859     }
860 }
```